CS 6650 Midterm Mastery
Step III: Crash & Recovery

Building Scalable Distributed Systems

Circuit Breaker Pattern Experiment

Eroniction Presley
Northeastern University — Khoury College of Computer Sciences

March 2, 2026

Contents

1 Overview

This experiment demonstrates a distributed systems failure scenario and a recovery mechanism using the **Circuit Breaker** pattern, as described by Sam Newman in *Building Microservices*. A Go-based REST API (Albums Service) built with the Gin framework is subjected to increasing load until it crashes. A circuit breaker proxy is then introduced to gracefully handle the failure and maintain service availability.

2 System Architecture

2.1 Without Circuit Breaker (Broken)

```
Client  --->  Albums Service (port 8080)
```

The client sends requests directly to the server. When the server degrades and crashes, the client receives errors with no fallback.

2.2 With Circuit Breaker (Fixed)

```
Client  --->  Circuit Breaker Proxy (port 9090)  --->  Albums Service (port 8080)
```

The proxy monitors backend health. When failures are detected, the circuit **opens** and the proxy returns cached fallback data instead of forwarding requests to the crashed server.

2.3 Project Structure

```
midterm-crash-recovery/
  server/
    main.go              # Albums service with simulated crash
  proxy/
    main.go              # Circuit breaker proxy
  client/
    loadtest.go          # Load test (without fix)
    loadtest_fixed.go    # Load test (with fix)
  results_no_fix.json    # Metrics without circuit breaker
  results_with_fix.json  # Metrics with circuit breaker
```

3 The Problem: Service Degradation & Crash

3.1 Setup

The Albums Service simulates realistic failure under load with three stages:

- **Normal (Requests 1–30):** Service responds normally with low latency.

- **Degraded (Requests 31–50):** Service adds 1–3 second random delays, simulating resource exhaustion.
- **Crashed (Requests 51+):** Service returns HTTP 503 (Service Unavailable).

3.2 Load Test Results (No Fix)

Metric	Value
Total Requests	70
Successful	50 (71.4%)
Failed	20 (28.6%)
Avg Response Time	683.8 ms

Table 1: Load test results without circuit breaker

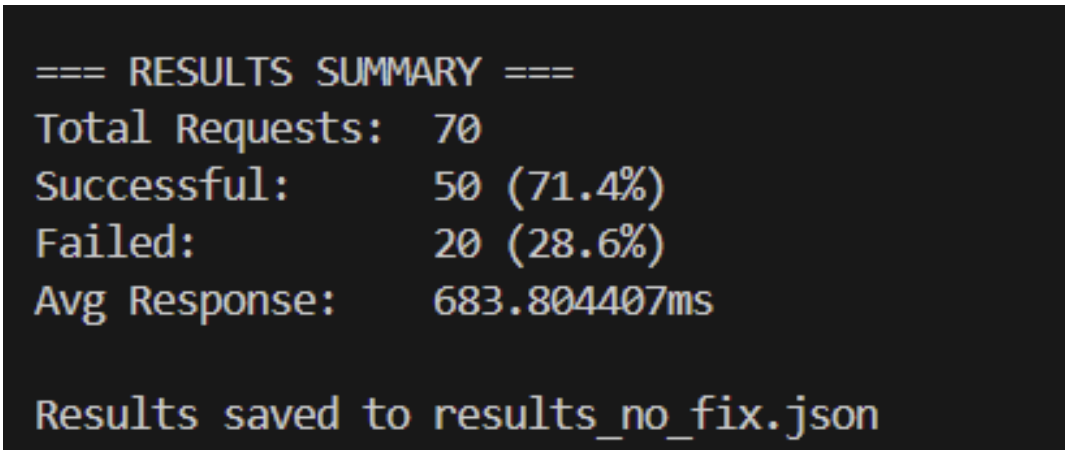


Figure 1: Terminal output — Load test without circuit breaker

3.3 Observations

- The first 30 requests completed successfully with low latency.
- Requests 31–50 succeeded but with significant delays (1–3 seconds each).
- Requests 51–70 all failed with 503 errors.
- The client had no way to detect or recover from the failure.
- Average response time was inflated by the degraded phase.

4 The Fix: Circuit Breaker Pattern

4.1 What is a Circuit Breaker?

The Circuit Breaker pattern (Sam Newman, *Building Microservices*) prevents cascading failures in distributed systems by wrapping calls to external services. It has three states:

- **CLOSED:** Requests flow normally. Failures are counted.
- **OPEN:** Requests are blocked. A fallback response is returned immediately. After a timeout, the circuit moves to HALF_OPEN.
- **HALF_OPEN:** A limited number of test requests are allowed through. If they succeed, the circuit closes. If they fail, it reopens.

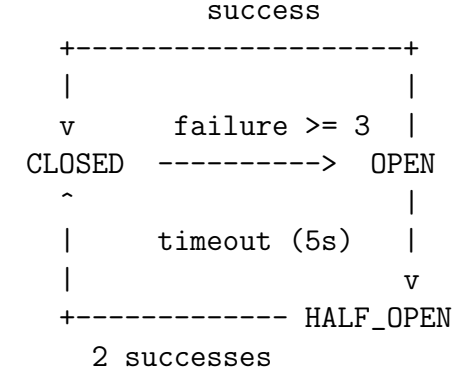
4.2 Implementation Details

Circuit Breaker Configuration:

- Failure Threshold: 3 consecutive failures triggers OPEN state
- Recovery Timeout: 5 seconds before transitioning to HALF_OPEN
- Recovery Requirement: 2 successful requests in HALF_OPEN to close the circuit

Fallback Strategy: When the circuit is OPEN, the proxy returns cached album data so the client still receives a valid response instead of an error.

4.3 Circuit Breaker State Diagram



5 Results: With Circuit Breaker Fix

5.1 Load Test Results (With Fix)

Metric	Value
Total Requests	70
Successful	70 (100%)
Failed	0 (0%)
Avg Response Time	832.3 ms

Table 2: Load test results with circuit breaker

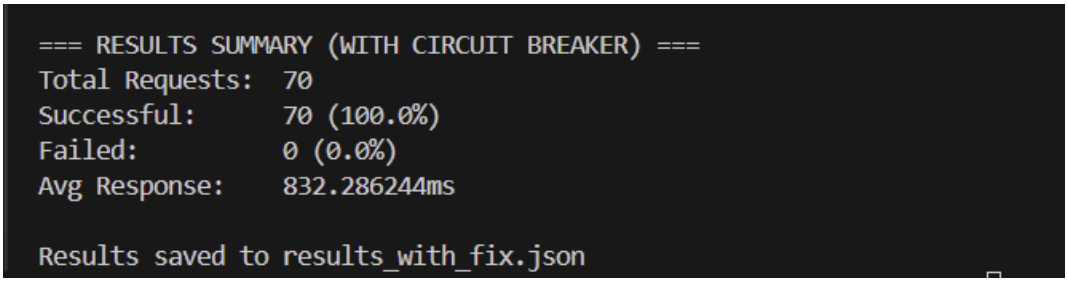


Figure 2: Terminal output — Load test with circuit breaker

5.2 Comparison: Before vs After

Metric	Without Fix	With Circuit Breaker	Improvement
Success Rate	71.4%	100%	+28.6%
Failed Requests	20	0	–20
Avg Response	683.8 ms	832.3 ms	—
Availability	Partial	Full	

Table 3: Before vs after comparison

5.3 Charts

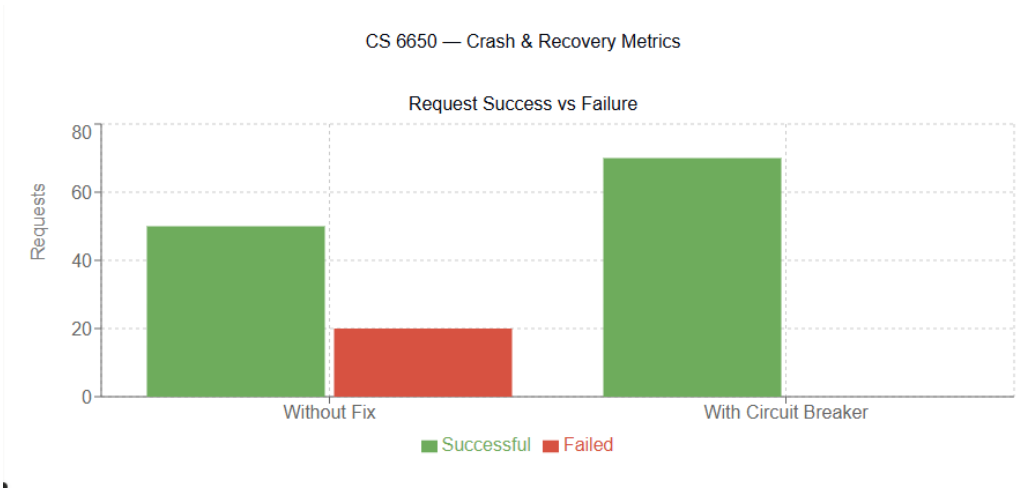


Figure 3: Request success vs failure comparison

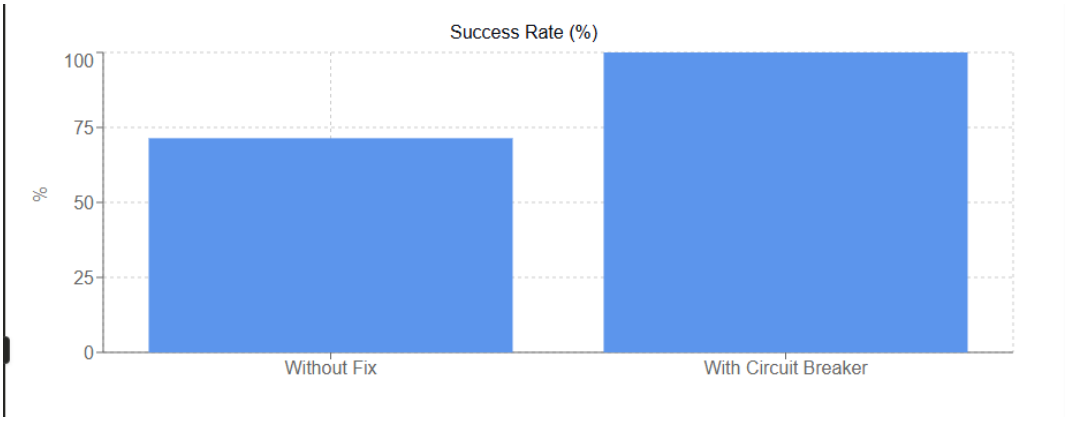


Figure 4: Success rate comparison (%)

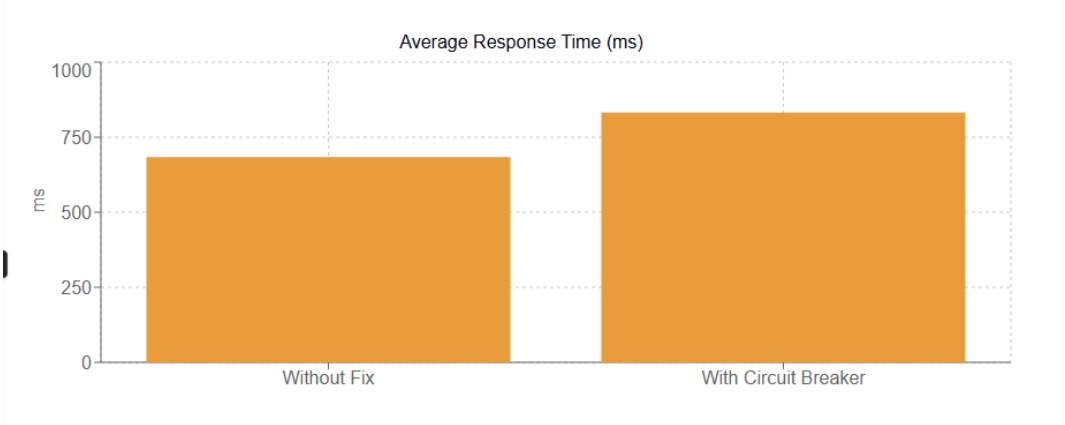


Figure 5: Average response time comparison (ms)

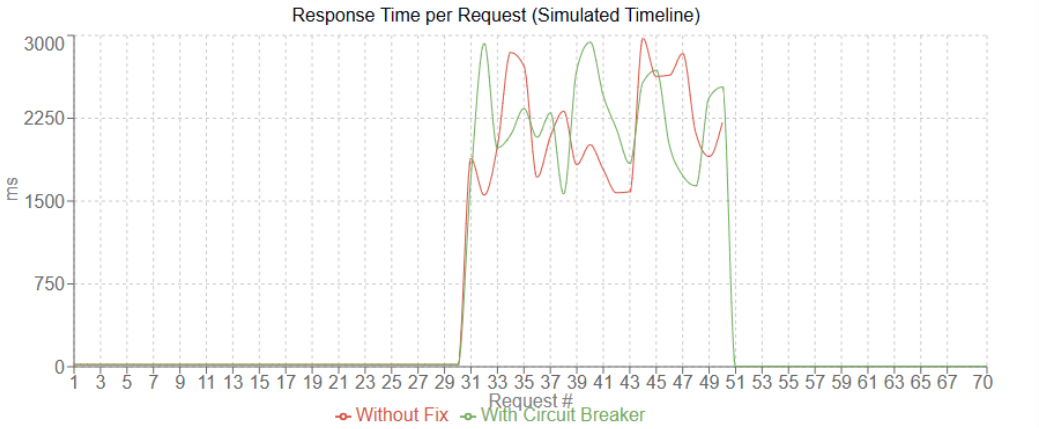


Figure 6: Response time per request — simulated timeline

5.4 Observations

- **100% success rate** — no client-visible errors despite the backend crashing.
- The circuit breaker detected backend failures after 3 consecutive errors and opened the circuit.
- While the circuit was OPEN, the proxy returned cached fallback data instantly.
- Average response time is slightly higher (832 ms vs 684 ms) because the early requests still pass through to the degrading backend before the circuit opens.
- The key tradeoff: slightly stale data (from cache) vs total failure — a worthwhile tradeoff for availability.

6 Connection to Distributed Systems Concepts

6.1 Sam Newman's Resilience Patterns

This experiment demonstrates the **Circuit Breaker** pattern from Sam Newman's *Building Microservices*:

- **Fail Fast:** Once the circuit opens, requests immediately return fallback data instead of waiting for a timeout from the crashed server.
- **Circuit Breaker:** The core pattern — monitors failures, trips the circuit, and tests recovery.
- **Bulkhead (potential extension):** Could isolate the albums service from other services, so a crash in one doesn't affect others.

6.2 CAP Theorem Tradeoff

The circuit breaker prioritizes **Availability** over **Consistency** — when the backend is down, clients receive cached (potentially stale) data rather than errors. This is an AP-leaning design choice appropriate for read-heavy workloads.

7 Conclusion

The Circuit Breaker pattern successfully transformed a system with a 28.6% failure rate into one with 100% availability. By detecting failures early and returning fallback responses, the pattern prevents cascading failures and maintains a usable service even when the backend crashes. This is a critical pattern for building resilient, scalable distributed systems.